PHASE 4 OF 8

Backend Core & Gemini Modules

A close look at main.py and the three Gemini-powered modules: Q&A, Summarizer and Learning Path.

main.py | qna.py | summary_module.py | learning_path.py

4.1 main.py — FastAPI Application

Purpose: the entry point of the whole app. It creates the FastAPI application, serves the frontend page, mounts static files, and defines every API route.

How it works

- `GET /` renders `templates/index.html`.
- `GET /qa?question=…` calls `answer_question_with_gemini()` from qna.py.
- `POST /explain` reads a JSON body with a *topic* field and calls `explain_topic()`.
- `POST /summarize` reads a *text* field and calls `summarize_text()`.
- `POST /quiz` reads a *text* field and calls `generate_quiz()`.
- `GET /learn/recommendations?topic=…` calls `get_learning_recommendations()`.
- Every POST route validates that the required field is not empty and returns a **400** error with a clear message if it is.

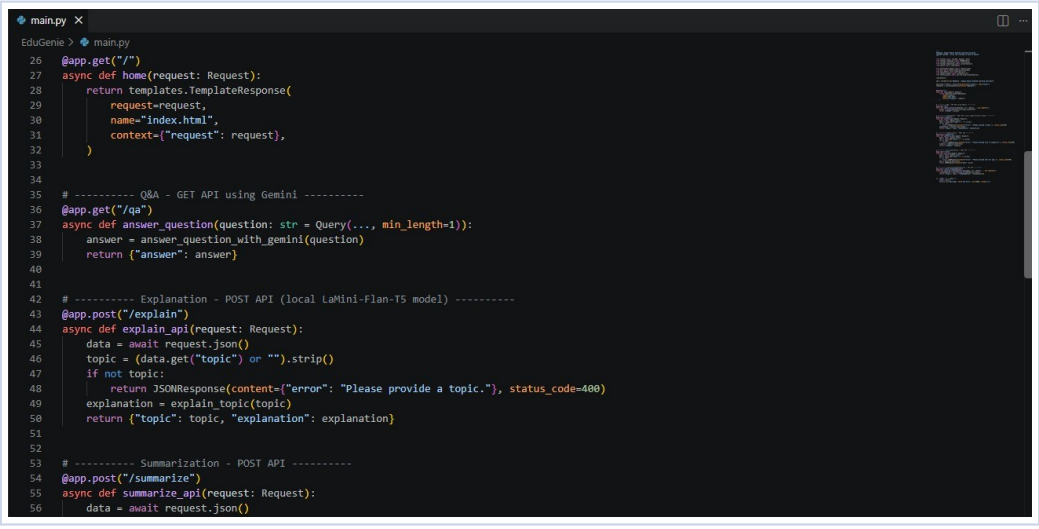


Figure — main.py: route validation for the /explain and /summarize endpoints.

Design note

Routes contain no AI logic. They validate input, delegate to one function, and wrap the result in JSON — which keeps `main.py` short and every feature independently replaceable.

4.2 qna.py — Question & Answer

Purpose: answers free-form questions using Gemini's chat interface.

- Loads `GEMINI_API_KEY` from the environment (via .env).
- Creates a Gemini chat session with `client.chats.create(…)` and sends the user's question.
- Returns the plain text of the model's reply, or a friendly error string if the key is missing or the call fails.

Simplified excerpt

```
def answer_question_with_gemini(question: str) -> str:
    if not GEMINI_API_KEY:
        return "Error: GEMINI_API_KEY is not set."
    chat = client.chats.create(model="gemini-3.6-flash")
    response = chat.send_message(question)
    return response.text
```

Calls: Google Gemini API.

4.3 summary_module.py — Summarizer

Purpose: condenses a longer passage into a short, plain-language summary.

- Builds a prompt instructing Gemini to "summarize the following text in simple language."
- Sends it through a Gemini chat session and returns the resulting text.

Simplified excerpt

```
def summarize_text(text: str) -> str:
    prompt = f"Summarize the following text in simple language:\n\n{text}"
    chat = client.chats.create(model="gemini-3.6-flash")
    response = chat.send_message(prompt)
    return response.text
```

Calls: Google Gemini API.

4.4 Shared Pattern Across Gemini Modules

Aspect	qna.py	summary_module.py	learning_path.py
Prompt style	Direct pass-through	Wrapped instruction	Tutor role + structure request
Return type	str	str	str
Missing-key handling	Friendly error string	Friendly error string	Friendly error string
Model	gemini-3.6-flash	gemini-3.6-flash	gemini-3.6-flash

4.5 learning_path.py — Learning Path Generator

Purpose: produces a structured, leveled roadmap (beginner → advanced) for learning a topic, with resource suggestions.

- Sends a prompt asking for key topics, learning order, and resources (videos, articles, books).
- Handles both possible Gemini response shapes (`response.text` or `response.parts`) before falling back to an error message.



Figure — learning_path.py: building the tutor prompt and reading the Gemini response.

4.6 Response-Handling Logic

Check	Result
`GEMINI_API_KEY` missing	Returns “Error occurred: GEMINI_API_KEY is not set. Add it to your .env file.”
`response.text` present	Returns the text directly
`response.parts` present	Returns the first part’s text
Neither available	Returns “Could not extract content from Gemini response.”
Any exception	Returns “Error occurred: <details>”

4.7 Observations & Recommendations

- **Centralize the model name.** `gemini-3.6-flash` is hard-coded in four modules; a single shared constant would make model changes a one-line edit.
- **Add request timeouts.** None of the Gemini-calling modules sets a timeout, so a slow API call can hang an endpoint.
- **Share one client.** Each module builds its own client; a common helper would remove duplication.